Lập trình mạng · Chương 5

Hiệu năng: lời gọi không chặn với CompletableFuture và gRPC async stub

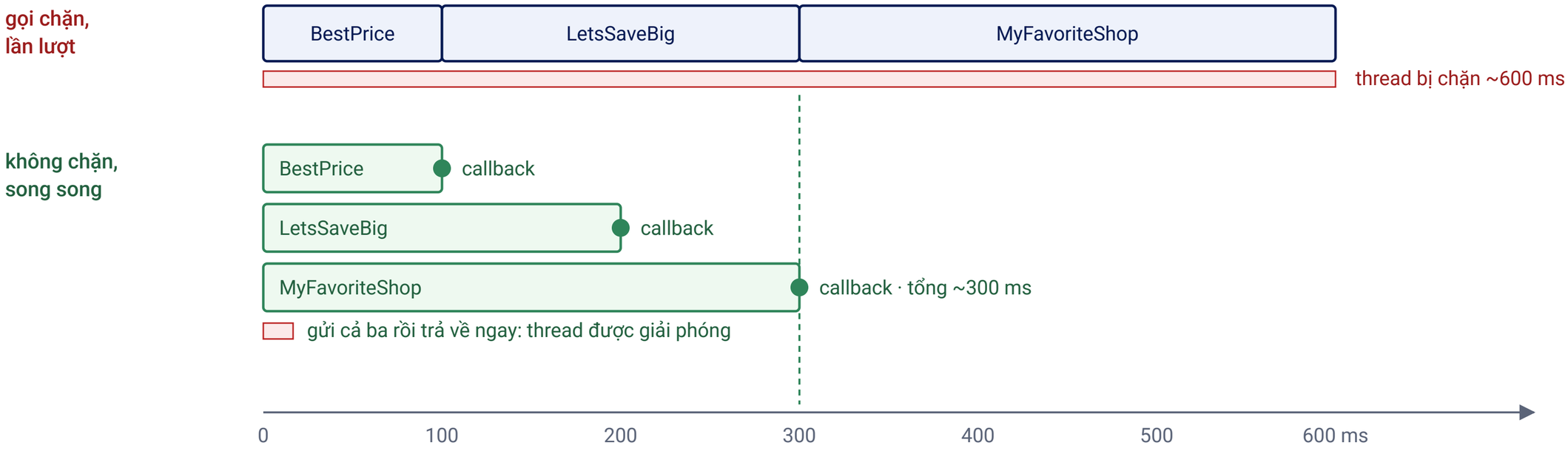
TS. Đặng Ngọc Hùng

Ba lời gọi nối đuôi nhau, thread chỉ chờ

1. Hỏi giá một sản phẩm ở **ba cửa hàng**, mỗi cửa hàng một server, rồi chọn **giá tốt nhất**
2. **Blocking stub** gọi lần lượt: thời gian là **tổng** ba lời gọi, thread chỉ chờ
3. Một cửa hàng lỗi làm hỏng cả yêu cầu

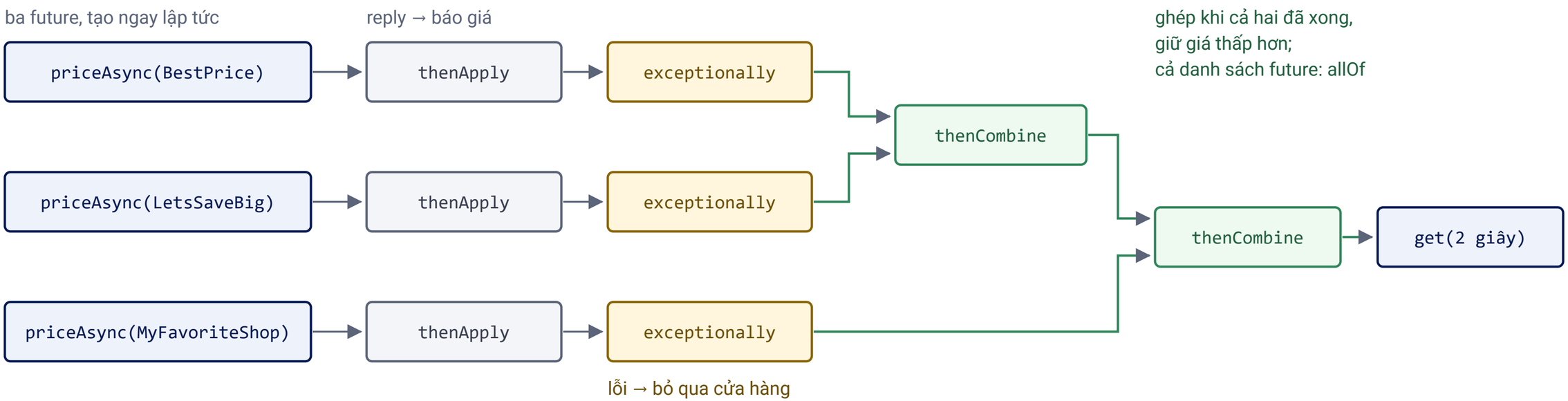
> **Câu hỏi của phần:** gửi cả ba lời gọi mà không chờ, rồi ghép kết quả và xử lý lỗi của từng lời gọi thế nào?

Lời gọi không chặn: thời gian chờ bằng lời gọi chậm nhất



Lời gọi không chặn trả về ngay, reply được xử lý qua **callback**; thời gian chờ chung bằng lời gọi **chậm nhất**.

CompletableFuture: nối bước xử lý, thay lỗi, ghép kết quả



CompletableFuture: `thenApply` biến đổi kết quả, `exceptionally` thay lỗi, `thenCombine` và `allOf` ghép kết quả; thread chỉ chờ một lần, ở `get`.

Async stub: gRPC gọi StreamObserver khi reply tới, callback hoàn tất future

```java
CompletableFuture<PriceReply> future = new CompletableFuture<>();
ShopGrpc.newStub(channel).getPrice(PriceRequest.newBuilder().setProduct(product).build(), new StreamObserver<>() {
    public void onNext(PriceReply reply) {
        future.complete(reply);  // callback: chạy trên thread của gRPC khi reply tới
    }
    public void onError(Throwable t) {
        future.completeExceptionally(t);
    }
});
return future;
```

- `newStub` tạo **async stub**: lời gọi trả về ngay sau khi gửi request
- gRPC gọi `onNext` hay `onError` trên thread của thư viện khi reply hoặc lỗi tới
- Callback chỉ hoàn tất future: phía gọi làm việc với **CompletableFuture**

Đo thời gian: hỏi lần lượt và hỏi song song ba cửa hàng

Java

```
CompletableFuture<Quote> bestPrice = quote(bestPriceShop, "BestPrice");
CompletableFuture<Quote> letsSaveBig = quote(letsSaveBigShop, "LetsSaveBig");
CompletableFuture<Quote> myFavorite = quote(myFavoriteShop, "MyFavoriteShop");
Quote cheapest = bestPrice.thenCombine(letsSaveBig, Quote::cheaper)  // ghép hai báo giá khi cả hai xong
        .thenCombine(myFavorite, Quote::cheaper)
        .get(2, TimeUnit.SECONDS);
```

Kết quả chạy

```
blocking  BestPrice 139.99
blocking  LetsSaveBig 129.99
blocking  MyFavoriteShop 124.99
blocking  best MyFavoriteShop 124.99, total 663 ms
async     best MyFavoriteShop 124.99, total 324 ms
```

Một cửa hàng lỗi: exceptionally thay lỗi, cửa hàng đó bị bỏ qua

Java

```
static CompletableFuture<Quote> quote(ManagedChannel channel, String shop) {
    return Prices.priceAsync(channel, PRODUCT)
            .thenApply(reply -> new Quote(shop, reply.getPrice(), null))
            .exceptionally(t -> Quote.unavailable(shop, t));  // lỗi → cửa hàng bị bỏ qua khi so giá
```

Kết quả chạy

```
blocking  BestPrice 139.99
blocking  LetsSaveBig 129.99
blocking  failed: UNAVAILABLE
blocking  total 346 ms
async     MyFavoriteShop skipped: UNAVAILABLE
async     best LetsSaveBig 129.99, total 222 ms
```

Callback chạy trên thread của thư viện: thao tác chặn chuyển sang executor riêng

```java
ExecutorService callbackThread = Executors.newSingleThreadExecutor(r -> new Thread(r, "grpc-callback"));
all[i] = ownExecutor
        ? reply.thenApplyAsync(CallbackDemo::applyDiscount, work)  // thao tác chặn chạy trên executor riêng
        : reply.thenApply(CallbackDemo::applyDiscount);            // thao tác chặn chạy ngay trên thread callback
```

Kết quả chạy

```
blocking work in callback       total 1632 ms
blocking work on own executor   total 817 ms
```

- Thao tác chặn trong callback giữ thread của thư viện: reply tới sau phải xếp hàng
- Tác vụ nặng, I/O chặn: chuyển sang **executor riêng** bằng `thenApplyAsync`

Câu hỏi 1

Ba lời gọi không chặn tới ba cửa hàng lần lượt mất 100 ms, 200 ms và 300 ms, rồi được ghép bằng phương thức thenCombine. Tổng thời gian chờ khoảng bao nhiêu?

A. 100 ms, bằng lời gọi nhanh nhất.

B. 200 ms, bằng thời gian trung bình.

C. 600 ms, bằng tổng ba lời gọi.

D. 300 ms, bằng lời gọi chậm nhất.

Câu hỏi 1

Ba lời gọi không chặn tới ba cửa hàng lần lượt mất 100 ms, 200 ms và 300 ms, rồi được ghép bằng phương thức thenCombine. Tổng thời gian chờ khoảng bao nhiêu?

A. 100 ms, bằng lời gọi nhanh nhất.

B. 200 ms, bằng thời gian trung bình.

C. 600 ms, bằng tổng ba lời gọi.

D. 300 ms, bằng lời gọi chậm nhất.

Ba lời gọi được gửi gần như cùng lúc và chờ song song; kết quả ghép chỉ có khi lời gọi chậm nhất xong. 600 ms là thời gian của cách gọi lần lượt.

Câu hỏi 2

Thư viện chạy mọi callback trên một thread duy nhất. Mỗi callback gọi một dịch vụ theo kiểu chặn, mất 500 ms. Ba reply tới gần như cùng lúc: điều gì xảy ra?

A. Reply thứ hai và thứ ba bị mất.

B. Thư viện tạo thêm thread để chạy song song các callback.

C. Các callback xếp hàng trên thread đó, tổng thời gian khoảng 1,5 giây.

D. Không ảnh hưởng gì, vì lời gọi đã là không chặn.

Câu hỏi 2

Thư viện chạy mọi callback trên một thread duy nhất. Mỗi callback gọi một dịch vụ theo kiểu chặn, mất 500 ms. Ba reply tới gần như cùng lúc: điều gì xảy ra?

A. Reply thứ hai và thứ ba bị mất.

B. Thư viện tạo thêm thread để chạy song song các callback.

C. Các callback xếp hàng trên thread đó, tổng thời gian khoảng 1,5 giây.

D. Không ảnh hưởng gì, vì lời gọi đã là không chặn.

Thread callback đang bận với thao tác chặn thì các reply khác chưa được xử lý; ba thao tác 500 ms nối nhau. Lời gọi không chặn chỉ giải phóng thread gọi, không làm callback hết chặn.

Tổng kết

1. **Giao tiếp bất đồng bộ**: gửi lời gọi rồi trả về ngay; thời gian chờ bằng lời gọi chậm nhất

2. **Async stub** báo reply qua **callback**; **CompletableFuture** nối bước, thay lỗi, ghép kết quả

3. Callback chạy trên thread của thư viện: thao tác chặn chuyển sang executor riêng

Từ khóa cần nhớ CompletableFuture · async stub · callback · giao tiếp bất đồng bộ

Mở rộng

Chủ đề nâng cao

- Reactive Streams và back-pressure
- Virtual thread như lựa chọn thay cho mã bất đồng bộ

Bài tập mở rộng

Hỏi giá ở 100 cửa hàng bằng async stub, giới hạn tối đa 10 lời gọi đang chạy.

Tiếp theo

Hiệu năng: NIO, Selector và mẫu Reactor